

RK0 Architecture Specification Document

Table of Contents

1. Purpose and architectural position	1
2. Logical layers and source organisation	2
3. Image construction, memory and startup	3
4. Scheduler representation and dispatch	4
4.1 Task control and queue membership	4
4.2 Dispatch boundary and architecture interface	5
5. Service entry, waiting and completion	5
6. Priority composition and propagation	6
7. Communication construction	8
7.1 Copied queues	8
7.2 Asynchronous ownership transfer	8
7.3 Synchronous endpoints	9
7.4 MRM publication	9
8. Time processing, interrupts and system work	10
9. Configuration, lifetime and cost bounds	11
9.1 Configuration and allocation	11
9.2 Cost model	11
10. Verification and controlled evolution	12
10.1 Existing executable evidence	12
10.2 Change map	13
Source register	13

1. Purpose and architectural position

REV:11SEP26/0801 | © 2026 www.antoniogiacomelli.com

This document explains how RK0 realises its functional contracts: component responsibilities, machine and task state, control paths, resource lifetimes and the invariants a change must preserve. The Functional Specification defines service meaning; this document provides the construction and change map. Contract IDs refer to that companion document.

RK0 is a single-core, statically linked, flat real-time executive. Its portable C services express coordination policy; a small Cortex-M layer constructs and switches execution contexts. Kernel and application share writable memory and privilege. Normal service calls are C calls on the calling task's stack. The startup SVC enters the first task; it is not a general syscall boundary. [D1, K1, K10]

The design puts recurring coordination semantics where the kernel can manage their scheduling consequences. Generic objects remain useful, while specialised services capture stable contracts such as rendezvous, invocation and most-recent publication. The system's commonality is the interaction between tasks; device drivers, payloads and application predicates remain variable. [B1, B2]

2. Logical layers and source organisation

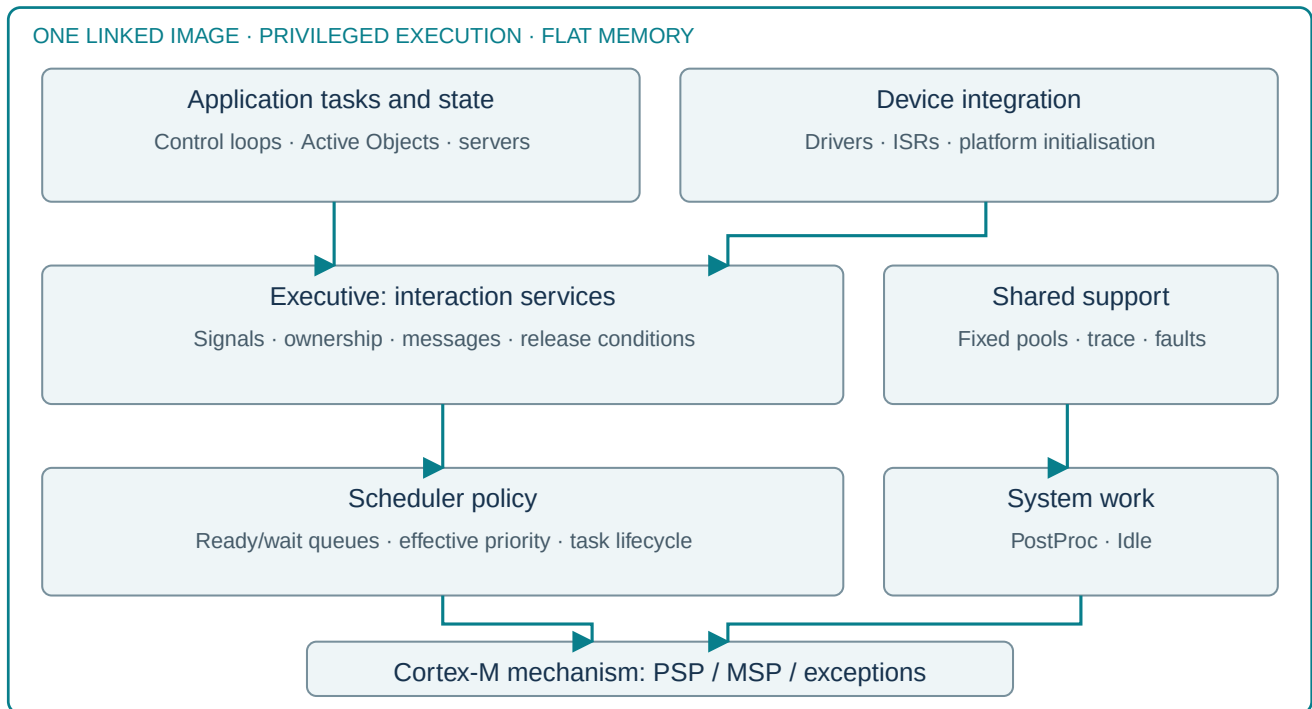


Figure 1. Figure A1. RK0 layers and responsibilities. All layers belong to one linked, privileged image.

The diagram is a responsibility view, not an access-control boundary. Application services can be ordinary functions, task-owned message endpoints, or coordinated shared state. An Active Object or driver server is an application arrangement of tasks and services. It does not require a distinct kernel object or a different dispatcher. [D1, B3]

Unit / source area	Responsibility and principal owned state
<code>core/inc/kapi.h</code>	Public declarations, convenience aliases and configuration-dependent API surface.
<code>core/inc/kcommondefs.h</code> , <code>kobjs.h</code>	Types, constants and object layouts shared by services and scheduler.
<code>core/src/ksch.c</code>	TCB pool, ready queues and bitmap, wait-queue helpers, task lifecycle, priority composition, dispatch policy and tick entry.
<code>ksema.c</code> , <code>kmutex.c</code> , <code>ksleepq.c</code> , <code>ktaskevents.c</code>	Token counts, mutex ownership, pure waiting and task event conditions.
<code>kmesgq.c</code> , <code>kringbuf.c</code>	Copied queues, mailbox variants and sender/receiver waits.
<code>kmesg.c</code> , <code>ksynchmesg.c</code> , <code>kmrm.c</code>	Direct ownership transfer, synchronous transaction state and most-recent buffer lifetimes.

Unit / source area	Responsibility and principal owned state
<code>ktimer.c</code> , <code>ksystasks.c</code>	Timeout lists, sleep calculations, callout scheduling and deferred work.
<code>kmem.c</code> , <code>kdynobjs.c</code>	Fixed-block allocation and per-type dynamic control-block pools.
<code>ktrace.c</code> , <code>kerr.c</code> , <code>kversion.c</code>	Optional observation, fault recording and version identity.
<code>arch/armv6m</code> or <code>arch/armv7m</code>	Stack frame construction, interrupt primitives, startup, SysTick/PendSV/SVC handlers and linker layout.
<code>app/</code> , root <code>Makefile</code>	Application hook, examples, board/emulator support and image construction.

Services can call scheduler helpers and one another where composition requires it. The logical layers do not prescribe an artificial chain of wrappers for each API call.

3. Image construction, memory and startup

F-LIFE-01, F-MEM-01, F-CTX-01. The build compiles portable kernel C, one architecture port and the selected application into an ELF image. The GNU Arm build uses GNU C99, architecture-specific assembly, section-level linking and an explicit linker script. BIN/HEX outputs are derived from the linked image. The library stubs in `syscalls.c` support the C runtime; their filename does not denote an RK0 privilege boundary. [K10, K12]

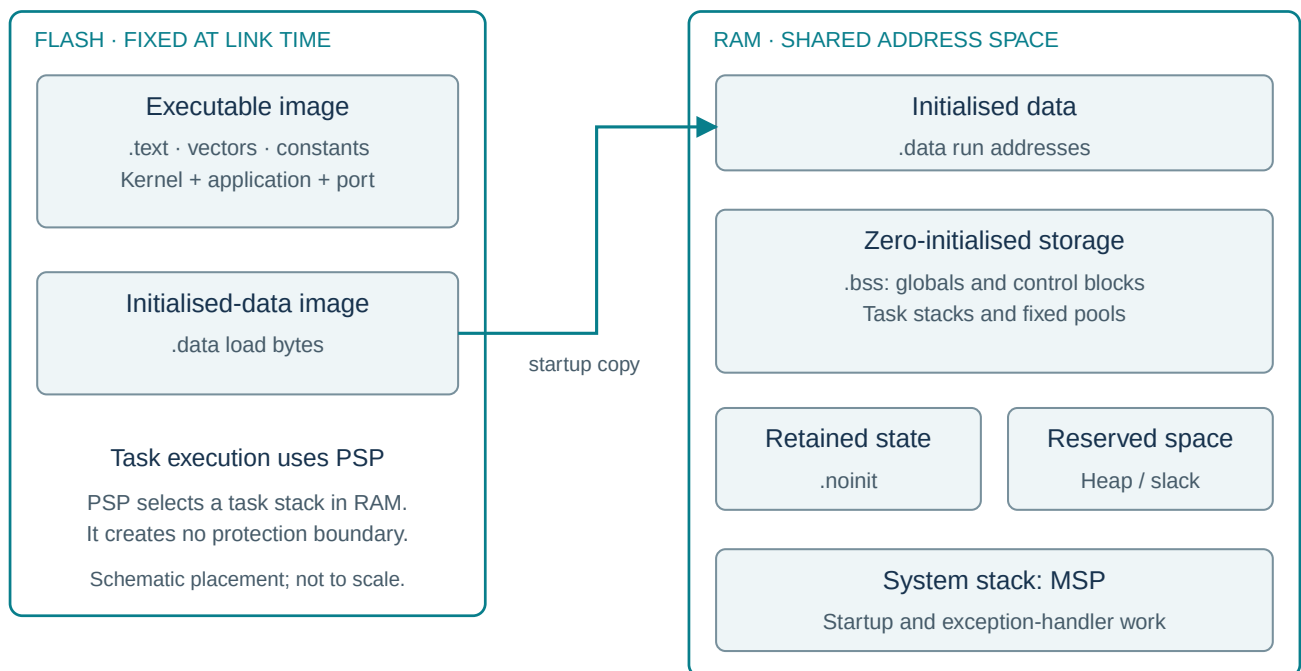


Figure 2. Figure A2. Storage placement and execution stacks. The layout is schematic, not to scale.

The linker places vectors, instructions and constants in flash; initialised data has a flash load image and RAM execution address. Startup copies `.data`, clears `.bss` and preserves `.noinit`. Task stacks and fixed pools are ordinary linked RAM storage. The MSP starts at the top of RAM for startup and exception handling; tasks use their own PSP-backed stacks. Heap reservation in a linker script does not make kernel allocation heap-based. [K9, K10, B4]

Startup proceeds as follows:

1. The architecture reset handler establishes the C runtime and enters application `main()`.
2. Platform setup establishes the clock and peripherals. `kCoreInit()` configures core exceptions and the tick source.
3. `kInit()` verifies version identity and initialises ready structures and the task pool, then invokes the application's `kApplicationInit()` hook.
4. The application initialises services and creates tasks. The first task-creation path also creates Idle and PostProc. Dynamic service-object pools are explicitly initialised when required.
5. `kInit()` initialises runtime timekeeping, readies initial tasks, selects the first context and invokes the startup SVC.
6. The SVC return starts privileged Thread mode on PSP. PostProc enables the configured SysTick counter as it starts its service loop.

All objects and buffers a task can use on first dispatch must already be valid. Runtime-spawned task stacks come from a predeclared partition. Reclaiming a running stack is deferred to PostProc. Static task termination is rejected. [K2]

4. Scheduler representation and dispatch

4.1 Task control and queue membership

The TCB joins machine context to execution and interaction state. Its principal groups are the saved stack pointer and exception return value; identity and lifecycle status; nominal/effective priority and scheduler-lock depth; release/timeout state; event bits; and the optional service relationships enabled at build time. [K2]

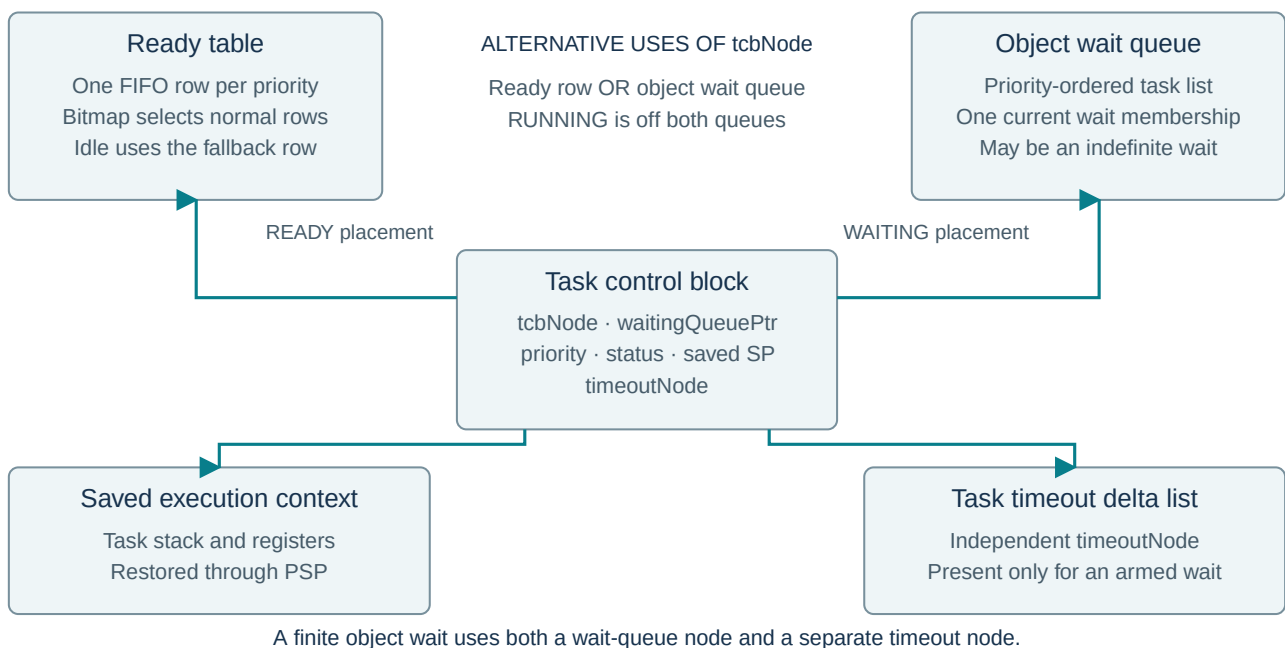


Figure 3. Figure A3. A TCB links execution state to one scheduling queue and an independent timeout node.

The intrusive `tcbNode` places a task in a ready or object-wait queue. The separate `timeoutNode` permits a waiting task to be simultaneously represented in a timeout list. `waitingQueuePtr` records the current object-wait membership independently of whether a finite timeout exists. This is essential for reprioritising indefinite waits.

At a stable scheduling boundary:

- A READY task is in the ready row indexed by its effective priority.
- A RUNNING task has been removed from its ready row.
- An object-waiting task has one recorded wait queue; a pure time or event-bit wait need not have one.
- An armed timeout node names exactly one timeout list through `listRefPtr`.
- A normal ready-priority bitmap bit is set exactly when its row is non-empty.

Kernel calls can have short intermediate states while interrupts are masked and a switch is pending. These intermediate states must not be mistaken for a second runnable membership.

F-SCH-01-04. The ready structure is an array of FIFO lists. Selecting the least-significant set bit gives the smallest numerical priority. ARMv7-M uses CLZ after isolating that bit; ARMv6-M uses a De Bruijn lookup. The default idle priority is 32: it has a dedicated ready row outside the normal 32-bit bitmap and is selected when that bitmap is zero. Thus the fallback is explicit while Idle still participates in queue operations. [K2, K10]

4.2 Dispatch boundary and architecture interface

`kPendCtxSwitch()` either requests PendSV or records a deferred switch according to the current status and scheduler-lock depth. Once PendSV executes, it saves the software register context, invokes `kSwch()`, restores the selected context and returns from the exception. `kSwch()` returns a still-RUNNING preempted task to its ready head, selects the next row and transfers the active scheduler-lock depth. The low-level handler marks the selected task RUNNING.

The Cortex-M hardware frame holds R0–R3, R12, LR, PC and xPSR; the software frame preserves R4–R11. The ARMv7-M path additionally handles configured floating-point context. TCB prefix offsets consumed by assembly are guarded by compile-time checks in the C scheduler. Moving those fields is an architecture-interface change, even if their C names remain unchanged. [K2, K10]

F-SCH-05. Scheduler locking suppresses task preemption, not interrupts. Its depth belongs to the task and is saved across a self-block. Kernel critical regions instead save PRIMASK, mask interrupts, and restore the saved value through `RK_CR_ENTER` / `RK_CR_EXIT`. Neither mechanism is a mutex replacement for arbitrary application ownership.

5. Service entry, waiting and completion

Most blocking services share a construction pattern: validate the operation, inspect state in a critical region, complete immediately if possible, otherwise record a wait reason and service context, arm a finite timeout if requested, enqueue the task and request a switch. When the call resumes, it interprets the release outcome and completes any remaining cleanup. [K2–K7]

F-WAIT-01, F-SCH-04. Wake and timeout paths must coordinate three distinct records: object-wait membership, timeout membership and ready membership. Removal from a wait queue must clear `waitingQueuePtr`; a successful early completion must disarm its timeout. Priority contributions affected by the released dependency must be removed before the task is placed at its final ready priority.

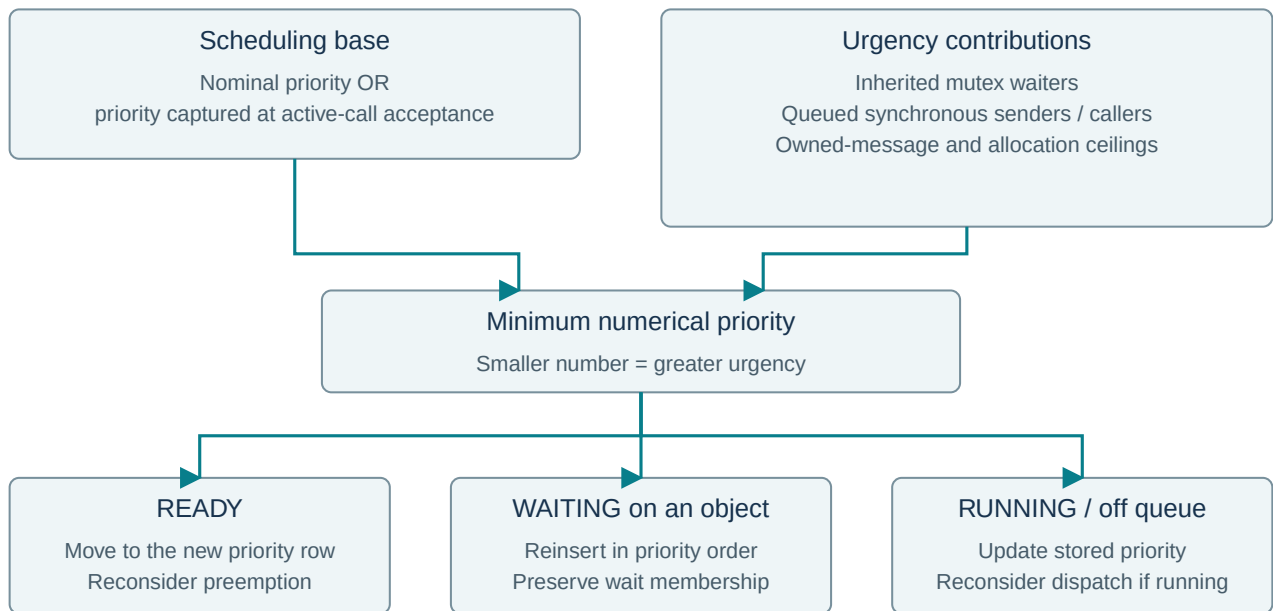
Wait kind	Stored context	Completion path
Semaphore, mutex, queue, Sleep Queue, async allocation	Object wait queue, status, optional timeout, service-specific output/ownership fields.	Object operation or generic blocking timeout; mutex timeout adds owner-priority cleanup.
Time sleep	TCB timeout node and sleep pseudo-state.	Time-event expiry readies the task without reporting a failed resource wait.
Task Event Register	Requested bits/options and event-sleep status.	Matching Set or event timeout; success consumes the requested mask.
Plain synchronous send/receive	Endpoint transaction fields and dedicated timeout type.	Copy or timeout helper removes the offer/receive destination and updates the endpoint.
Invocation	Queued/active/abandoned call state, server relation and optional call timeout.	Reply or call-timeout helper; active timeout retains an abandoned transaction until closure.

`RK_WAITING_ALLOC` is a wait reason, not a new scheduling class. The allocation path uses the same membership helpers and generic blocking timeout as other object waits. Its priority ceiling is recognised from the allocation context and wait-queue pointer, not by a universal numeric test of the status code. Trace maps it to `ALLOC`. [K2, K3, K6, K11]

Condition-variable helpers illustrate a deliberate composition. `kCondVarWait()` retains a scheduler lock while unlocking the mutex and entering the Sleep Queue. A self-block permits another task to run with its own lock state. The helper later reacquires the mutex indefinitely before releasing its owned scheduler-lock depth. The predicate remains application state. A finite wait does not bound that reacquisition; see F-CV-01. [K4]

6. Priority composition and propagation

Priority calculation is centralised in `kTaskCalcEffectivePrio()`. Individual services change dependencies and request recomputation instead of independently restoring nominal priority. This is the main architectural connection between service semantics and scheduler ordering. [K2]



Automatic cross-task propagation follows inherited-mutex wait edges.

Figure 4. Figure A4. Priority calculation and application to task state. Adoption changes the base; other contributions increase urgency.

The calculation starts at `prioNominal`. An ACTIVE accepted call substitutes `synchMesgActiveCallerPrio`. It then takes the minimum numerical value across the heads of inherited mutex wait queues, queued synchronous senders/callers, message pools represented in the owned-message list, and a pending allocation pool. ABANDONED call state does not substitute a base.

F-PRIO-01. `kTaskUpdateEffectivePrio()` applies the result according to task placement:

Placement	Update operation
READY	Remove from the old row, change priority, append to the new row, update bitmap and reconsider preemption.
Object wait queue	Remove through the membership helper, change priority and reinsert by priority. Equal-priority insertion is FIFO.
RUNNING	Change priority and reconsider the highest ready competitor.
Other off-queue state	Update stored priority; subsequent readiness uses that value.

`kTaskUpdateEffectivePrioChain()` follows a changed task to the owner of an inherited mutex on which it is blocked. It stops when priority is unchanged or no such mutex edge exists. Reordering a blocked task before following the edge ensures the next owner sees the correct highest waiter.

The same path handles propagation and removal. Releasing one mutex does not discard contributions from other mutexes or message pools. A message ceiling can raise a task already waiting on a mutex, and that urgency can then propagate through the mutex chain. The public benches exercise these combinations. [K12]

The scope is precise: this helper does not traverse arbitrary synchronous-message dependency chains. An accepted caller's priority is a snapshot taken at acceptance. Endpoint operations

explicitly update sender/caller contributions at enqueue, receipt, accept, reply and timeout. A new protocol that needs another propagation edge must define it explicitly, including its removal path and cycle assumptions.

RK0's mutex policy uses actual owner/waiter relations. It does not implement original PCP's global admission rule. The optional asynchronous pool ceiling is a separate admission and ownership contribution and introduces no global mutex system ceiling. [B5, K2, K6]

7. Communication construction

7.1 Copied queues

F-MSGQ-01-03. A Message Queue combines a fixed-word ring buffer with separate priority-ordered sender and receiver lists. A successful send either copies directly to a waiting receiver or writes a ring slot. A receive consumes a slot and makes capacity available to a waiting sender. Ring message order and task-wait order are separate structures.

Single-slot overwrite and broadcast reuse the queue representation. Broadcast tracks the current receiver cohort using `broadcastReceivers` and waiter metadata. While that cohort owns the slot, normal receives and overwrite cannot consume it. Multi-receiver wake work can be deferred. Send callbacks execute inside the originating operation's critical region, so their cost belongs to that operation's interrupt-latency budget. [K5]

7.2 Asynchronous ownership transfer

F-ASY-01-03. Each message contains pool identity, sender/receiver identity, state, a queue node and a distinct owner-list node. Each endpoint is TCB state. The queue node represents delivery order; the owner node represents lifetime and ceiling responsibility. Those roles must not be collapsed. [K6]

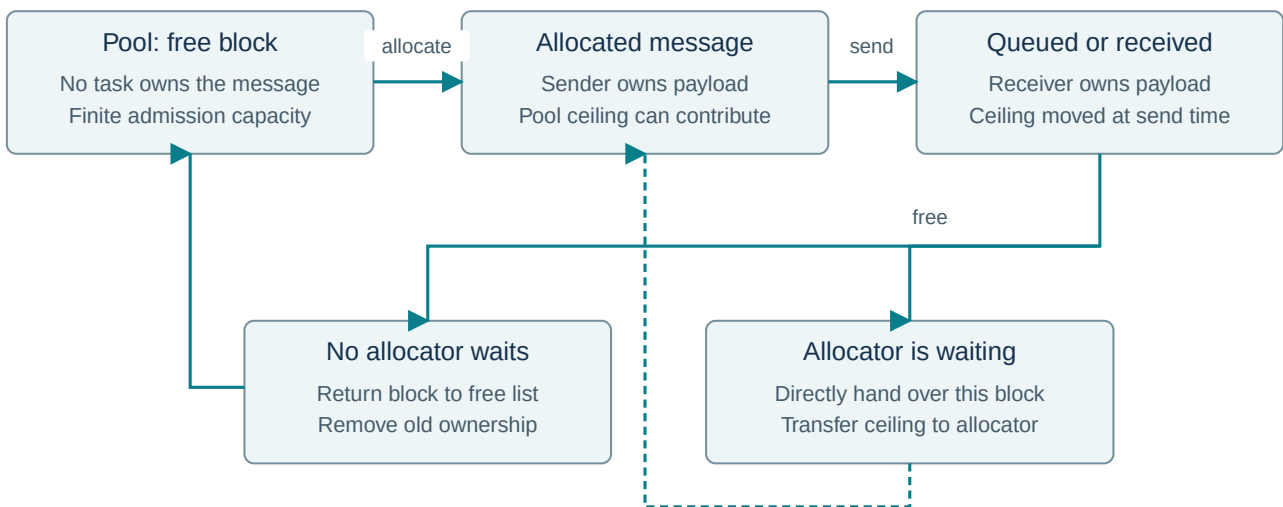


Figure 5. Figure A5. Asynchronous message ownership and pool reuse. The send operation transfers ownership before consumption.

`kMsgSetOwner_()` is the ownership transfer point and recomputes the old and new owners. The

sender owns an allocated message; send transfers ownership to the receiver even if it is queued. Receive changes QUEUED to RECEIVED without another ownership transfer. Free either returns the block to its partition or hands it directly to a waiting allocator, transferring the ceiling without an intervening unowned free-list state.

The allocation wait records `asynchMsgAllocDestPtr` so a release can supply the block directly. A finite timeout removes the wait membership and therefore its wait-ceiling contribution before readiness; the resumed call clears its remaining output context. Sender-filtered receive scans the endpoint queue and preserves unmatched messages. Endpoint capacity is bounded by available message blocks rather than by a second queue-slot reservation.

7.3 Synchronous endpoints

F-SYNC-01. Plain rendezvous records a sender's buffer pointer and byte count in its TCB. The receiver endpoint maintains pending senders and a promoted offer. Receipt copies from the offering task's still-valid buffer. Timeout removes that offer, promotes the next pending sender when necessary and recomputes receiver urgency. The buffer is not independently copied into a kernel backlog. [K7]

F-SYNC-02–04. Invocation adds a caller queue and one active-caller relation to the server endpoint:

Transaction state	Data and scheduling action
QUEUED	Caller retains request/reply descriptors; server can inherit queued urgency. The call timeout is already armed.
ACTIVE	Accept copies the request, removes caller-queue membership, records the active relation and captures the adopted base. The caller remains blocked through reply.
Completed	Reply copies to the caller, disarms the call timeout, clears both transaction relations, recomputes server priority and readies the caller.
ABANDONED	Active timeout clears live reply-write destinations and removes adoption. Reply later closes the relation without copying to timed-out caller storage.

A higher-priority queued caller contributes urgency while the server finishes the active request. It does not preempt that request at the protocol level. Reply completion restores the nominal base plus surviving contributions; the next Accept establishes the next transaction's base. [K2, K7]

7.4 MRM publication

F-MRM-01–03. MRM uses descriptor and payload partitions, a current-buffer pointer, a reservation flag and reader counts. Reserve selects a free descriptor/payload pair or reuses an unreferenced current buffer. Publish copies outside the metadata critical section while the buffer is reserved, then commits the current pointer and clears the reservation. Get increments the reader count before copying outside the critical section. Unget releases obsolete storage when the count reaches zero. [K8]

Reservation and pinning separate lengthy payload copying from short metadata operations. A reserved current buffer is not readable, so Get returns null during that reuse interval. Reader references protect copies already in progress. The protocol relies on single-publisher use and a bounded number of outstanding reader tokens; it is not an unbounded version store.

8. Time processing, interrupts and system work

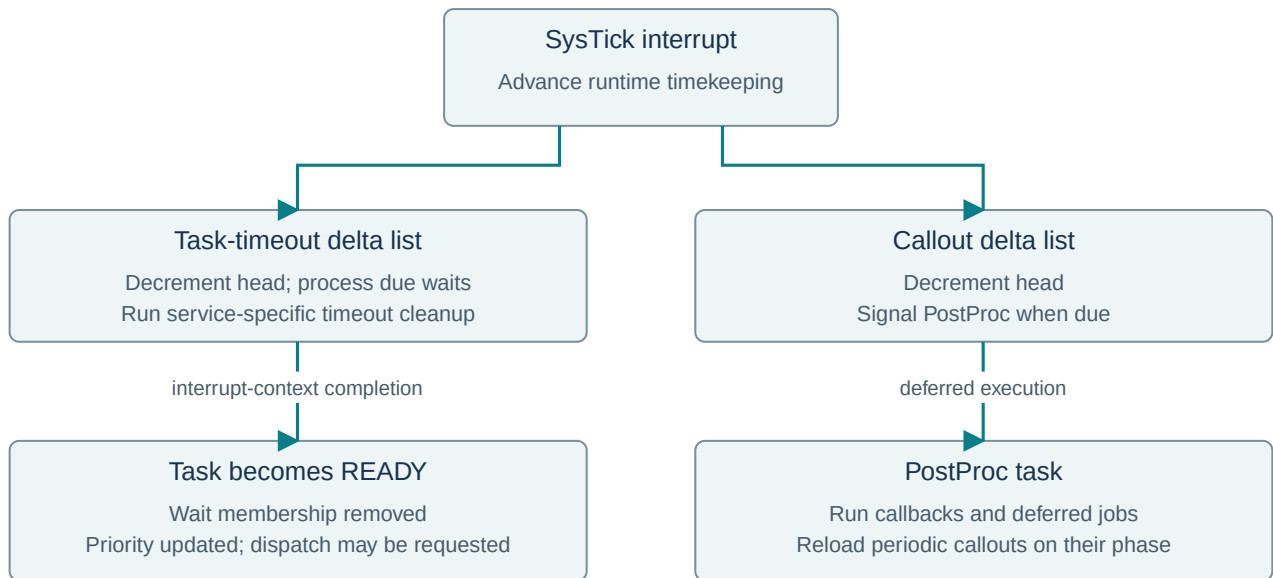


Figure 6. Figure A6. Task timeouts and deferred callouts take different execution paths.

F-TIME-01-05, F-WAIT-01. Task waits and callout timers use separate delta lists: `RK_gTimeOutListHeadPtr` and `RK_gTimerListHeadPtr`. A node stores the delay relative to its predecessor. Insertion subtracts preceding deltas; removal transfers its remaining delta to its successor. `ListRefPtr` identifies the list and permits head removal without searching. [K3]

SysTick increments runtime timekeeping and decrements the task-timeout head. It processes all now-expired task nodes, invoking the appropriate wake or transaction cleanup. For callouts, it advances the separate timer list and signals PostProc when its head expires. PostProc executes callbacks and schedules reloads on their phase grid. Callout execution is therefore subject to deferred-service latency.

PostProc is created at priority 0 with `RK_NO_PREEMPT`. It handles a bounded deferred-job ring, including multi-waiter wake/reset work and deferred task destruction. Each pass through the job drain has a budget equal to the ring capacity, which is `RK_NTHREADS` in this baseline. Callout draining is a separate loop over due timers. The application must account for total due callback work as high-priority non-preemptible task demand. [K10]

The ring bounds storage; it does not make deferred completion instantaneous. Enqueue failure is an explicit outcome. Application and kernel objects referenced by queued jobs must remain alive until execution. Idle executes WFI when no normal-priority task is ready. Trace can introduce an additional task and UART interactions; these consume task and timing budgets when enabled.

9. Configuration, lifetime and cost bounds

9.1 Configuration and allocation

`kconfig.h` selects optional semaphore, mutex, Sleep Queue, message, MRM, callout, dynamic-object and trace facilities. It also defines capacities and timing parameters. Relevant dependency groups include asynchronous direct messages with message-queue support, and condition helpers with Mutex and Sleep Queue support. Every translation unit must use a compatible configuration because options change structures as well as functions. [K1]

`kmem.c` threads a free list through unused fixed-size blocks. Allocation removes its head; release adds a block to the head. Partition initialisation and control-block storage are explicit. Dynamic non-task services allocate from per-type control-block arrays; payload storage is passed separately to their Create API. These services reuse the same initialisation and operational implementation as caller-supplied objects. [K9]

Destroy checks pool provenance and service-specific quiescence. Mutex destruction rejects owners/waiters; queue destruction rejects waiting participants and an active broadcast; MRM destruction rejects reserved or reader-held versions. Timer destruction cancels its schedule before reclaiming the control block. Task destruction additionally checks endpoint dependencies and obtains an off-stack reclamation context when necessary.

These are cooperative lifetime rules within a trusted image. Handles are pointers, and clearing one handle variable does not revoke aliases. Introducing stronger lifetime validation or memory protection would change the object model and its access costs.

9.2 Cost model

Let W be the number of waiters examined, A the active timeout nodes, E the expiries processed in one tick, M the mutexes owned by a task, H its owned messages, Q the queued direct messages, and L a dependency-chain length. These are configured or application-bounded quantities, not universal constants. [K2–K10]

Operation	Dominant structural cost
Choose next ready priority; ready insertion/removal	$O(1)$ for the fixed-width bitmap and known intrusive node.
Priority-ordered wait insertion or reprioritisation	$O(W)$ to locate position; known-node unlink is $O(1)$.
Timeout insertion / removal	$O(A)$ insertion; $O(1)$ removal of a known node.
Tick with task expiries	$O(1)$ head decrement plus E wake/cleanup operations and their priority costs.
Effective-priority recomputation	Scans M owned mutexes and H owned messages, plus endpoint queue heads; applying it can require $O(W)$ reinsertion.

Operation	Dominant structural cost
Transitive mutex update	Sum of recomputation and placement costs along up to L affected task relationships.
Fixed partition allocation / release	O(1) free-list update; enabled double-free validation scans the free list on release.
Copied queue / synchronous transfer / MRM copy	Proportional to the configured payload length; queue sizes are limited to 1/2/4/8 words.
Filtered asynchronous receive	Up to O(Q) to locate a matching sender.
Broadcast, reset, lifecycle cleanup	Depends on affected waiters, references and configured capacities.

O(1) choose-next does not imply O(1) for every kernel call or a fixed interrupt-latency bound. Analysis must include critical-section duration, tracing, callbacks, simultaneous expiries and priority-chain traversal. Hardware timing claims require a named board, clock, compiler, configuration and measurement boundary.

10. Verification and controlled evolution

10.1 Existing executable evidence

The public Makefile provides `public-qemu-benches` for ARMv6-M and ARMv7-M. It builds and runs five targeted examples and checks their pass markers. The CI workflow invokes those benches for both ports, in addition to Cppcheck and externally supplied private unit/scheduler harnesses. Public workflow configuration is not evidence that every wiki requirement has been exercised on every revision. [K12, W2]

Public example	Contract behaviour exercised
<code>06_transitive_priority_inheritance.c</code>	Mutex-chain propagation and restoration.
<code>07_wait_queue_repriority.c</code>	Effective-priority changes and order while blocked.
<code>08_async_ceiling_wait.c</code>	Pool allocation waits, ceiling contributions, timeout/handoff and send-time transfer.
<code>09_ready_queue_repriority.c</code>	READY-owner movement and priority restoration, including timeout.
<code>10_extended_rendezvous_priority.c</code>	Active-call adoption, queued-caller contribution and restoration.

Representative commands, from the repository root:

```
make ARCH=armv7m qemu
make ARCH=armv6m qemu
make ARCH=armv7m public-qemu-benches
make ARCH=armv6m public-qemu-benches
```

The hardware build packages named in the baseline README are separate 0.74.0 distributions. Their version labels must be preserved when describing those packages; they are not automatically a hardware validation of this 0.80.1 source snapshot. [K12]

10.2 Change map

Proposed change	Invariants and companion material to revise
New wait reason or release path	Membership/timeout cleanup, readiness, termination eligibility, priority contribution removal and trace name; F-WAIT/F-LIFE.
New priority protocol	Contribution source, base versus boost, propagation edge, removal path and READY/WAITING placement; F-PRIO.
Communication change	Completion point, copy/ownership boundary, saturation, payload lifetime and cancellation after partial progress; F-MSGQ/F-ASY/F-SYNC/F-MRM.
TCB or port change	Assembly offsets, saved registers, PSP geometry, exception return, critical-section restoration and both architecture builds; F-SCH/F-CTX.
Capacity or feature change	Struct layout consistency, pool sizing, system/trace task demand, worst-case loops and configuration combinations; F-MEM.
Timing or diagnostic change	Expiry bursts, callout latency, non-preemptible work, trace cost and measured build identity; F-TIME/F-DIAG.

Preserve a clear distinction between intended contract, implementation evidence and regression evidence. A focused change should update the corresponding FSD clause, this construction map and the smallest regression that observes the changed behaviour. Stable IDs make this possible without turning the specifications into copies of the source tree.

Source register

Code references K1–K12 resolve to commit [77bf7b1906edf1dfd8391816dcfac99b4d3885c4](#). DocBook and wiki revisions are recorded separately. Blog pages were consulted on 12 September 2026.

D1 — [RK0 DocBook 0801.0.0.1](#), system 0.80.1; docs revision d13a608e03e4.

K1 — Public surface and configuration: [kapi.h](#), [kcommondefs.h](#), [kconfig.h](#).

K2 — Scheduler and task representation: [ksch.c](#); [kobjs.h](#).

K3 — Time and waits: [ktimer.c](#); [ktimer.h](#).

K4 — Shared-state services: [ksema.c](#), [kmutex.c](#), [ksleepq.c](#), [ktaskevents.c](#).

K5 — Copied communication: [kmesgq.c](#); [kringbuf.c](#).

- K6** — Asynchronous direct messages: [kmesg.c](#).
- K7** — Synchronous rendezvous and invocation: [ksynchmesg.c](#).
- K8** — Most-recent publication: [kmrm.c](#).
- K9** — Allocation and dynamic objects: [kmem.c](#); [kdynobjs.c](#).
- K10** — Machine layer and system work: [ARMv6-M](#); [ARMv7-M](#); [ksystasks.c](#); [syscalls.c](#).
- K11** — Diagnostics and identity: [ktrace.c](#), [kerr.c](#), [kversion.c](#).
- K12** — Build and regression evidence: [Makefile](#); [CI workflow](#); [public examples](#); [README.md](#).
- W2** — [Requirements Mapping](#), wiki snapshot 85f43fb49554. Test names are traceability references.
- B1** — [If Everything Is an Event, Nothing Is](#).
- B2** — [RK0: a Software Engineering Perspective](#).
- B3** — [Active Objects in RK0](#).
- B4** — [How RK0 Gets Up and Running](#).
- B5** — [Why RK0 Chose Not to Implement PCP](#).